

A view of 0 or 1 elements: `views::maybe`

Document #: P1255R7
Date: 2022-05-09
Project: Programming Language C++
Audience: SG9, LEWG
Reply-to: Steve Downey
<sdowney2@bloomberg.net, sdowney@gmail.com>

Abstract: This paper proposes `views::maybe` a range adaptor that produces a view with cardinality 0 or 1 which adapts copyable object types, values, and nullable types such as `std::optional` and pointer to object types.

§ 1 Changes

§ 1.1 Changes since R6

- Extend to all object types in order to support list comprehension
- Track working draft changes for Ranges
- Add discussion of *borrowed_range*
- Add an example where pipelines use references.
- Add support for proxy references (explore `std::pointer_traits`, etc).
- Make `std::views::maybe` model `std::ranges::borrowed_range` if it's not holding the object by value.
- Add a const propagation section discussing options, existing precedent and proposing the option that the author suggests.

§ 1.2 Changes since R5

- Fix reversed before/after table entry
- Update to match C++20 style [[N4849](#)] and changes in Ranges since [[P0896R3](#)]
- `size` is now `size_t`, like other ranges are also
- add synopsis for adding to `<ranges>` header
- Wording clean up, formatting, typesetting
- Add implementation notes and references

§ 1.3 Changes since R4

- Use `std::unwrap_reference`
- Remove conditional `noexceptness`
- Adopted the great concept renaming

§ 1.4 Changes since R3

- Always Capture
- Support `reference_wrapper`

§ 1.5 Changes since R2

- Reflects current code as reviewed
- Nullable concept specification

Remove `Readable` as part of the specification, use the useful requirements from `Readable`

- Wording for `views::maybe` as proposed-
- Appendix A: wording for a `view_maybe` that always captures

§ 1.6 Changes since R1

- Refer to `views::all` Behavior of capture vs refer is similar to how `views::all` works over the expression it is given
- Use wording ‘range adaptor object’ Match current working paper language

§ 1.7 Changes since R0

- Remove customization point objects

Removed `views::maybe_has_value` and `views::maybe_value`, instead requiring that the nullable type be dereferenceable and contextually convertible to `bool`.

- Concept `Nullable`, for exposition

Concept `Nullable`, which is `Readable` and contextually convertible to `bool`

- Capture rvalues by decay copy

Hold a copy when constructing a view over a nullable rvalue.

- Remove `maybe_view` as a specified type

Introduced two exposition types, one safely holding a copy, the other referring to the nullable

§ 2 Before / After Table

Before	After
<pre> { auto&& opt = possible_value(); if (opt) { // a few dozen lines ... use(*opt); // is *opt OK ? } } </pre>	<pre> for (auto&& opt : views::maybe(possible_value())) { // a few dozen lines ... use(opt); // opt is OK } </pre>
<pre> std::optional o{7}; if (o) { *o = 9; std::cout << "o=" << *o << " prints 9\n"; } std::cout << "o=" << *o << " prints 9\n"; </pre>	<pre> std::optional o{7}; for (auto&& i : views::maybe(std::ref(o))) { i = 9; std::cout << "i=" << i << " prints 9\n"; } std::cout << "o=" << *o << " prints 9\n"; </pre>
<pre> std::vector<int> v{2, 3, 4, 5, 6, 7, 8, 9, 1}; auto test = [](int i) -> std::optional<int> { switch (i) { case 1: case 3: case 7: case 9: return i; default: return {}; } }; auto&& r = v ranges::views::transform(test) ranges::views::filter([](auto x){return bool(x);}) ranges::views::transform([](auto x){return *x;}) ranges::views::transform([](int i) { std::cout << i; return i; }); </pre>	<pre> std::vector<int> v{2, 3, 4, 5, 6, 7, 8, 9, 1}; auto test = [](int i) -> std::optional<int> { switch (i) { case 1: case 3: case 7: case 9: return i; default: return {}; } }; auto&& r = v ranges::views::transform(test) ranges::views::transform(views::maybe) ranges::views::join ranges::views::transform([](int i) { std::cout << i; return i; }); </pre>

§ 3 Motivation

In writing range transformation it is useful to be able to lift a value into a view that is either empty or contains the value. For types that are nullable, constructing an empty view for disengaged values and providing a view to the underlying value is useful as well. The adapter `views::single` fills a similar purpose for non-nullable values, lifting a single value into a

view, and `views::empty` provides a range of no values of a given type. The type `views::maybe` can be used to unify single and empty into a single type for further processing. This is in particular useful when translating list comprehensions.

```
std::vector<std::optional<int>> v{
    std::optional<int>{42},
    std::optional<int>{},
    std::optional<int>{6 * 9}};

auto r = views::join(views::transform(v, views::maybe));

for (auto i : r) {
    std::cout << i; // prints 42 and 54
}
```

In addition to range transformation pipelines, `views::maybe` can be used in range based for loops, allowing the nullable value to not be dereferenced within the body. This is of small value in small examples in contrast to testing the nullable in an if statement, but with longer bodies the dereference is often far away from the test. Often the first line in the body of the if is naming the dereferenced nullable, and lifting the dereference into the for loop eliminates some boilerplate code, the same way that range based for loops do.

```
{
    auto&& opt = possible_value();
    if (opt) {
        // a few dozen lines ...
        use(*opt); // is *opt OK ?
    }
}

for (auto&& opt : views::maybe(possible_value())) {
    // a few dozen lines ...
    use(opt); // opt is OK
}
```

The view can be on a `std::reference_wrapper`, allowing the underlying nullable to be modified:

```
std::optional o{7};
for (auto&& i : views::maybe(std::ref(o))) {
    i = 9;
    std::cout << "i=" << i << " prints 9\n";
}
std::cout << "o=" << *o << " prints 9\n";
```

Of course, if the nullable is empty, there is nothing in the view to modify.

```
auto oe = std::optional<int>{};
for (int i : views::maybe(std::ref(oe)))
    std::cout << "i=" << i << '\n'; // does not print
```

Converting an optional type into a view can make APIs that return optional types, such as lookup operations, easier to work with in range pipelines.

```

std::unordered_set<int> set{1, 3, 7, 9};

auto flt = [=](int i) -> std::optional<int> {
    if (set.contains(i))
        return i;
    else
        return {};
};

for (auto i : ranges::iota_view{1, 10} | ranges::views::transform(flt)) {
    for (auto j : views::maybe(i)) {
        for (auto k : ranges::iota_view(0, j))
            std::cout << '\a';
        std::cout << '\n';
    }
}

// Produce 1 ring, 3 rings, 7 rings, and 9 rings

```

§ 4 Lazy monadic pythagorean triples

Eric Niebler's pythagorean triple example, using current C++ and proposed `views::maybe`.

```

// "and_then" creates a new view by applying a
// transformation to each element in an input
// range, and flattening the resulting range of
// ranges. A.k.a. bind
// (This uses one syntax for constrained lambdas
// in C++20.)
inline constexpr auto and_then = [](auto&& r, auto fun) {
    return decltype(r)(r) | std::ranges::views::transform(std::move(fun)) |
        std::ranges::views::join;
};

// "yield_if" takes a bool and a value and
// returns a view of zero or one elements.
inline constexpr auto yield_if = [](bool b, auto x) {
    return b ? maybe_view{std::move(x)}
        : maybe_view<decltype(x)>{};
};

void print_triples() {
    using std::ranges::views::iota;
    auto triples = and_then(iota(1), [](int z) {
        return and_then(iota(1, z + 1), [=](int x) {
            return and_then(iota(x, z + 1), [=](int y) {
                return yield_if(x * x + y * y == z * z,
                    std::make_tuple(x, y, z));
            });
        });
    });

    // Display the first 10 triples
    for (auto triple : triples | std::ranges::views::take(10)) {
        std::cout << '(' << std::get<0>(triple) << ',' << std::get<1>(triple)

```

```

        << ',', << std::get<2>(triple) << ')' << '\n';
    }
}

```

The implementation of `yield_if` is essentially the type unification of `single` and `empty` into `maybe`, returning an empty on false, and a range containing one value on true.

§ 5 Proposal

Add a range adaptor object `views::maybe`, returning a view over an object, capturing by value. For nullable objects, provide a zero size range for objects which are disengaged. A *nullable* object is one that is both contextually convertible to `bool` and for which the type produced by dereferencing is an equality preserving object. Non void pointers, `std::optional`, and the proposed expected [P0323R9] types all model *nullable*. Function pointers do not, as functions are not objects. Iterators do not generally model *nullable*, as they are not required to be contextually convertible to `bool`.

§ 6 Design

The basis of the design is to hybridize `views::single` and `views::empty`. If the view is over a value that is not nullable it is like a single view if constructed with a value, or is of size zero otherwise. For nullable types, if the underlying object claims to hold a value, as determined by checking if the object when converted to `bool` is true, begin and end of the view are equivalent to the address of the held value within the underlying object and one past the underlying object. If the underlying object does not have a value, begin and end return `nullptr`.

§ 7 Borrowed Range

A `borrowed_range` is one whose iterators cannot be invalidated by ending the lifetime of the range. For `views::maybe`, the iterators are `T*`, where `T` is essentially the type of the dereferenced nullable. For raw pointers and `reference_wrapper` over nullable types, the iterator for `maybe_view` points directly to the underlying object, and thus matches the semantics of `borrowed_range`. This means that `maybe_view` is conditionally borrowed. A `maybe_view<shared_ptr>`, however, is not a borrowed range, as it participates in ownership of the `shared_ptr` and might invalidate the iterators if upon the end of its lifetime it is the last owner.

An example of code that is enabled by borrowed ranges, if unlikely code:

```

num = 42;
int k = *std::ranges::find(views::maybe(&num), num);

```

Providing the facility is not a significant cost, and conveys the semantics correctly, even if the simple examples are not hugely motivating. Particularly as there is no real implementation impact, other than providing template variable specializations for `enable_borrowed_range`.

§ 8 List Comprehension Desugaring

The case for having `maybe_view` available is seen in desugaring list comprehensions, where they naturally show up in guard clauses.

Looking at Haskell for a [formal lowering of comprehensions](#):

We write `Desugar [ e | Q ]` to mean the desugaring of the comprehension `[ e | Q ]`

Expressions: `e`

Lists of qualifiers: `Q,R,S`

– Basic forms

`Desugar [ e | ] = return e`

`Desugar [ e | p <- e, Q ] = e >>= \p -> Desugar [ e | Q ]`

`Desugar [ e | e, Q ] = guard e >> \p -> Desugar [ e | Q ]`

Where:

`>>=` is the normal bind operator

`(>>=) :: m a -> (a -> m b) -> m b`

equivalent to

`\x -> join (fmap f x)`

See the `abd_then` function above.

`>>` is a bind that sequences but discards the left side

`(>>) :: m a -> m b -> m b`

defined as

`k >> f = k >>= (\_ -> f)`

`guard` has the type `guard :: Alternative f => Bool -> f ()` and is defined as

```
guard True  = return ()
guard False = empty
```

See the `yield_if` above.

`return` is a constructor for a monad over `T`, lifting a value into the monad. `return :: Monad m => t -> m t`

The pythagorean triple example above is a typical hand desugaring of a list comprehension.

```
[(x,y,z) | z <- [1..x], y <- [1..z], x <- [1..y], x*x + y*y == z*z]
```

The guard function functions as a filter. It's usually possible to rewrite the guard and join into a filter function, but fusing the guard conditions may not be straightforward.

§ 9 Implementation

A publically available implementation at https://github.com/steve-downey/view_maybe based on the Ranges implementation in libstdc++. There are no particular implementation difficulties or tricks. The declarations are essentially what is quoted in the Wording section and the implementations are described as *Effects*.

[Compiler Explorer Link to Before/After Examples](#)

§ 10 Wording

§ 10.1 Synopsis

Modify 26.2 Header synopsis

```
// [range.maybe], maybe view
template<copy_constructible T>
    requires see below
class maybe_view;

template <typename T>
constexpr inline bool enable_borrowed_range<maybe_view<T*>> = true;

template <typename T>
                                constexpr                inline                bool
                                enable_borrowed_range<maybe_view<reference_wrapper<T*>>> = true;

namespace views { inline constexpr unspecified maybe = unspecified; }
```

§ 10.2 Maybe View [range.maybe]

§ 10.2.1 Overview

- 1 maybe_view produces a view over a *nullable* that is either empty if the *nullable* is empty, or provides access to the contents of the *nullable* object.
- 2 The name `views::maybe` denotes a customization point object ([customization.point.object]). For some subexpression *E*, the expression `views::maybe(E)` is expression-equivalent to:
 - `maybe_view{E}`, the view specified below, if the expression is well formed, where `decay-copy(E)` is moved into the `maybe_view`
 - otherwise `views::maybe(E)` is ill-formed.

[Note: Whenever `views::maybe(E)` is a valid expression, it is a prvalue whose type models view. — end note]

3 [Example:

```
optional o{4};
maybe_view m{o};
for (int i : m)
    cout << i;           // prints 4
```

— end example]

§ 10.2.2 Concept *nullable*

1 Types that:

- are contextually convertible to `bool`
- are dereferenceable
- have `const` references which are dereferenceable
- the `iter_reference_t` of the type and the `iter_reference_t` of the `const` type, will :
 - satisfy `is_lvalue_reference`
 - satisfy `is_object` when the reference is removed
 - for `const` pointers to the referred to types, satisfy `convertible_to`
- or are a `reference_wrapper` around a type that satisfies *nullable*

model the exposition only *nullable* concept

2 Given a value `i` of type `T`, `T` models *nullable* only if the expression `*i` is equality-preserving.
[Note: The expression `*i` is required to be valid via the exposition-only *nullable* concept).
— end note]

3 For convenience, the exposition-only *is-reference-wrapper-v* is used below.

```
// For Exposition
template <typename T>
struct is_reference_wrapper : false_type {};

template <typename T>
struct is_reference_wrapper<reference_wrapper<T>> : true_type {};

template <typename T>
inline constexpr bool is_reference_wrapper_v
    = is_reference_wrapper<T>::value;
```

```
// For Exposition
template <class Ref, class ConstRef>
concept readable_references =
    is_lvalue_reference_v<Ref> &&
    is_object_v<remove_reference_t<Ref>> &&
    is_lvalue_reference_v<ConstRef> &&
    is_object_v<remove_reference_t<ConstRef>> &&
    convertible_to<add_pointer_t<ConstRef>,
                  const remove_reference_t<Ref>*>;
```

```

template <class T>
concept nullable =
    is_object_v<T> &&
    requires(T& t, const T& ct) {
        bool(ct); // Contextually bool
        *t; // T& is deferenceable
        *ct; // const T& is deferenceable
    }
    && readable_references<iter_reference_t<T>, // Ref
                          iter_reference_t<const T>>; // ConstRef

template <class T>
concept wrapped_nullable =
    is-reference-wrapper-v<T>
    && nullable<typename T::type>;

```

§ 10.2.3 Class template *maybe_view*

```

namespace std::ranges {
    template <copy_constructible Maybe>
        requires (nullable<Maybe> || wrapped_nullable<Maybe>)
    class maybe_view : public view_interface<maybe_view<Maybe>> {
    private:
        using T = /* see below */
        copyable_box<Maybe> value_; // exposition only (see [range.copy.wrap])

    public:
        constexpr maybe_view() = default;
        constexpr explicit maybe_view(Maybe const& maybe);

        constexpr explicit maybe_view(Maybe&& maybe);

        template<class... Args>
        requires constructible_from<Maybe, Args...>
        constexpr maybe_view(in_place_t, Args&&... args);

        constexpr T* begin() noexcept;
        constexpr const T* begin() const noexcept;
        constexpr T* end() noexcept;
        constexpr const T* end() const noexcept;

        constexpr size_t size() const noexcept;

        constexpr T* data() noexcept;
        constexpr const T* data() const noexcept;
    };
}

```

```

// For Exposition
using T = std::remove_reference_t<
    iter_reference_t<typename unwrap_reference_t<Maybe>>>;

```

```
constexpr explicit maybe_view(Maybe const& maybe);
```

- 1 *Effects:* Initializes value_ with maybe.

```
constexpr explicit maybe_view(Maybe&& maybe);
```

- 2 *Effects*: Initializes `value_` with `std::move(maybe)`.

```
template<class... Args>
constexpr maybe_view(in_place_t, Args&&... args);
```

- 3 *Effects*: Initializes `value_` as if by `value_{in_place, forward<Args>(args)...}`.

```
constexpr T* begin() noexcept;
constexpr const T* begin() const noexcept;
```

- 4 *Effects*: Equivalent to: `return data();`.

```
constexpr T* end() noexcept;
constexpr const T* end() const noexcept;
```

- 5 *Effects*: Equivalent to: `return data() + size();`.

```
static constexpr size_t size() noexcept;
```

- 6 *Effects*: Equivalent to:

```
if constexpr (is-reference-wrapper-v<Maybe>) {
    return bool(value_.get().get());
} else {
    return bool(value_.get());
}
```



```
constexpr T* data() noexcept;
```

- 7 *Effects*: Equivalent to:

```
Maybe& m = *value_;
if constexpr (is-reference-wrapper-v<Maybe>) {
    return m.get() ? addressof(*(m.get())) : nullptr;
} else {
    return m ? addressof(*m) : nullptr;
}
```

```
constexpr const T* data() const noexcept;
```

- 8 *Effects*: Equivalent to:

```
const Maybe& m = *value_;
if constexpr (is-reference-wrapper-v<Maybe>) {
    return m.get() ? addressof(*(m.get())) : nullptr;
} else {
    return m ? addressof(*m) : nullptr;
}
```

§ 11 Impact on the standard

A pure library extension, affecting no other parts of the library or language.

§ 12 References

[N4849] Richard Smith. 2020-01-14. Working Draft, Standard for Programming Language C++.

<https://wg21.link/n4849>

[P0323R9] JF Bastien, Vicente Botet. 2019-08-03. std::expected.

<https://wg21.link/p0323r9>

[P0896R3] Eric Niebler, Casey Carter, Christopher Di Bella. 2018-10-07. The One Ranges Proposal.

<https://wg21.link/p0896r3>